

School of Computer Science and Artificial Intelligence

Lab Assignment # 10.2

Program : B. Tech (CSE)

Specialization :AIML

Course Title : AI Assisted Coding

Course Code : 23CS002PC304

Semester : VI

Academic Session : 2025-2026

Name of Student : k.sathwika

Enrollment No. : 2303A52096

Batch No. : 33

Date :17/02/26

TASK_01: Error Detection and Correction

Task:

Use AI to analyze a Python script and correct all syntax and logical errors.

PROMPT:

Analyze the following Python code, identify syntax and logical errors, and provide corrected executable code with explanations.

Sample Input Code:

```
def calculate_total(nums)
sum = 0
for n in nums
sum += n
return total
```

CORRECTED CODE:

```
def calculate_total(nums):
    total_sum = 0
    for n in nums:
        total_sum += n
    return total_sum
```

```
# Test the corrected function
numbers = [1, 2, 3, 4, 5]
result = calculate_total(numbers)
print(f"The total of {numbers} is: {result}")
```

ERRORS IDENTIFIED:

1. Missing colon : after function definition.
2. Missing colon : in the for loop.
3. Variable name mismatch: sum was used but total returned.
4. sum shadows built-in Python function sum() (bad practice).

OUTPUT:

```
... The total of [1, 2, 3, 4, 5] is: 15
```

TASK_02: Code Style Standardization

Task:

Use AI to refactor Python code to comply with standard coding style

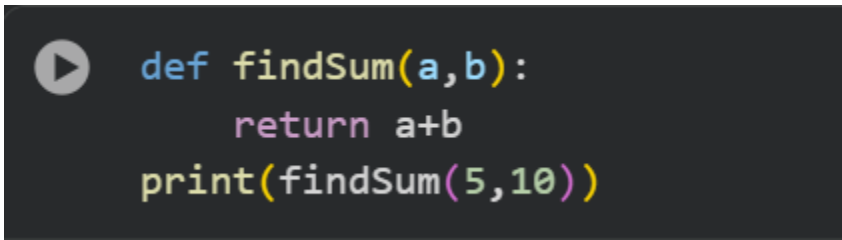
PROMPT USED:

Refactor the Python code to follow PEP 8 style guidelines and proper formatting.

SAMPLE CODE:

```
def findSum(a,b):return a+b  
print(findSum(5,10))
```

CORRECTED CODE:

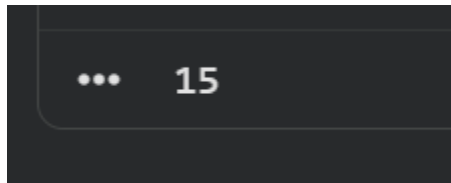


```
def findSum(a,b):  
    return a+b  
print(findSum(5,10))
```

Improvements Made

- Function renamed to snake_case (find_sum) as per PEP 8.
- Proper indentation and spacing.
- Assigned result to a variable for clarity.

OUTPUT:



TASK_03: Code Clarity Improvement

Task:

Use AI to improve code readability without changing its functionality

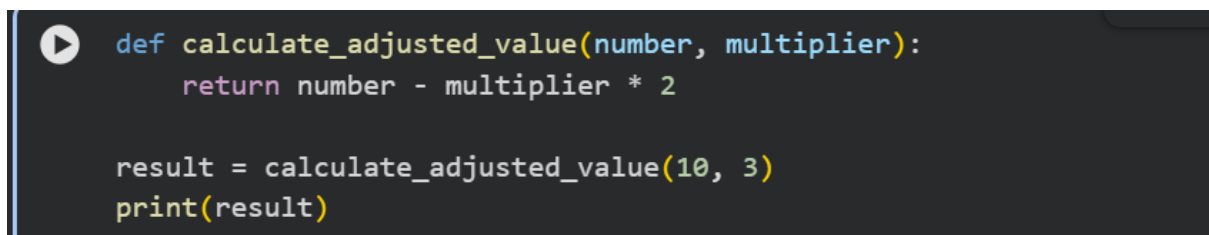
PROMPT:

Improve readability with meaningful names and proper indentation without changing functionality

SAMPLE CODE:

```
def f(x,y):  
    return x-y*2  
print(f(10,3))
```

CORRECTED CODE:

A screenshot of a code editor with a dark background. On the left side, there is a play button icon. The code is written in a light blue font and shows a function definition and its usage with improved readability and indentation.

```
def calculate_adjusted_value(number, multiplier):  
    return number - multiplier * 2  
  
result = calculate_adjusted_value(10, 3)  
print(result)
```

IMPROVEMENTS:

- . Renamed function and variables with meaningful names.
- Proper indentation.
- Added descriptive naming for clarity.

OUTPUT:



Task 4 – Structural Refactoring (Reusable Functions)

PROMPT:

“Refactor repetitive code into reusable functions to improve modularity.”

Sample Code:

```
print("Hello Ram")  
print("Hello Sita")  
print("Hello Ravi")
```

CORRECTED CODE:

```
def greet(name):  
    print(f"Hello {name}")  
  
greet("Ram")  
greet("Sita")  
greet("Ravi")
```

Improvements

- Eliminated repetition using a reusable function.
- Improved maintainability and scalability.

OUTPUT:

```
... Hello Ram  
     Hello Sita  
     Hello Ravi
```

Task_05:

Efficiency Enhancement

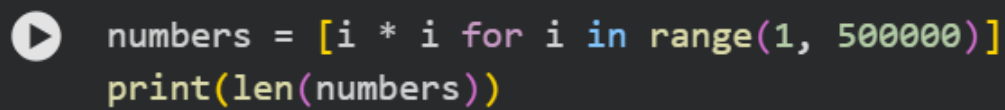
Prompt Used

“Optimize the Python code for better performance while keeping functionality unchanged.”

Sample Input Code

```
numbers = [ ]  
for i in range(1, 500000):  
    numbers.append(i * i)  
print(len(numbers))
```

OPTIMISED CODE:

A dark-themed code editor snippet showing two lines of Python code. The first line is `numbers = [i * i for i in range(1, 500000)]` and the second line is `print(len(numbers))`. The code is color-coded: `numbers` is white, `=` is white, `[` is yellow, `i` is white, `*` is white, `i` is white, `for` is purple, `i` is white, `in` is blue, `range` is green, `(1, 500000)` is green, `]` is yellow, `print` is white, `(` is white, `len` is green, `(` is white, `numbers` is white, `)` is white, and `)` is white.

```
numbers = [i * i for i in range(1, 500000)]  
print(len(numbers))
```

OUTPUT:

A dark-themed output console snippet showing the result of the code execution. It displays three dots followed by the number 499999.

```
... 499999
```

IMPROVEMENTS:

- Replaced loop + append with list comprehension.
- Faster execution and more Pythonic.
- Reduced code length and improved readability.

